

MariposaMarket

Una compra explicable, una confirmación confiable.

6 de septiembre de 2026 · Decisiones adoptadas para la prueba · Implementación con apoyo de IA

0. Contexto y diseño

Tomé **Grupo Mariposa como referencia** para MariposaMarket: compra B2B, paleta azul/turquesa y CTA amarillo. Prioricé un resumen con líneas, origen de descuentos, obsequios, impuestos y total. Descarté tarjetas de altura fija y texto truncado: los componentes adaptan el contenido, protegen controles del teclado y barras del sistema, y escalan las imágenes con el mismo criterio.

2. La prioridad difícil: precios

Exploré las prioridades **con IA, simulando las perspectivas de una encargada de Marketing y una Product Owner**. Contrasté claridad de beneficios y consistencia de compra; esa exploración orientó mi decisión:

Combo → escala → obsequios → impuestos → total → crédito

El combo reserva unidades; la escala usa el saldo y acumula por familia, asignando tramos por SKU ascendente para independizar el resultado del orden del carrito. Respeto prioridades e incompatibilidades, con dinero entero y reglas por país. **Confirmando el snapshot:** descarté recalcular para conservar el precio aceptado.

4. ERP y notificaciones PUSH

Separé **API y worker**; la API confirma aun sin broker. Un tópico alimenta **dos suscripciones: ERP y PUSH**, con dummies y ACK independiente. Cada fallo espera 1/2/4/8/16 s, sin ACK. Tras cinco reintentos, persisto la DLQ en Mongo antes del ACK y envío correo a Mailpit. Descarté el consumidor conjunto y el conteo aproximado del broker: prioricé independencia y presupuesto durable.

6. Mi revisión de código

Si lo hubiera escrito otra persona, revisaría aislamiento por tenant/país, overflow, redondeo, tramos por familia, atomicidad y carreras de idempotencia. Exigiría pruebas de respuesta perdida, reinicio del broker y fallo entre efecto externo y registro local. Revisaría accesibilidad y controles con teclado. **La revisión llevó a dividir funciones complejas y actualizar dependencias;** mantengo esos controles en CI para prevenir regresiones.

1. Stack y uso de IA

Elegí **Go por mis años de experiencia**, flexibilidad y curva de aprendizaje ya recorrida; aprender otro backend habría restado foco al dominio. Frente a Angular, elegí Flutter por el ecosistema Google y su alcance multidispositivo. Acepté aprenderlo, aunque no lo conocía. **Desarrollé la solución con apoyo de IA** y validé con pruebas; esa asistencia no sustituye mi responsabilidad de revisar el código.

3. Una intención, un pedido

Descarté confiar solo en deshabilitar el botón. El frontend persiste la clave antes de enviar y la reutiliza tras recargas. **Tenant + clave + hash** permiten replay o rechazo de contenido distinto. Una transacción Mongo guarda pedido, débito condicional, idempotencia y outbox, e impide confirmar dos veces la cotización. El 201 llega después del commit: pedido consultable y débito único.

5. Despliegue y alcance consciente

Elegí **Compose** con Go, Mongo replica set, Pub/Sub emulado y Flutter Web/nginx; preparé documentación para Pages. Dejé fuera autenticación productiva, ERP/PUSH reales y builds nativos para concentrar la prueba en precios y consistencia, con adaptadores simulados durables. Postergué despliegue cloud y operación avanzada: requieren infraestructura y contratos reales. La entrega incluye CI de pruebas, sintaxis, complejidad y seguridad.

7. Riesgo y mitigación

Riesgo: el receptor acepta un efecto y el worker cae antes de guardar éxito. Exijo deduplicar **evento:destino**. La reconciliación conserva IDs y excluye destinos en DLQ. Si SMTP falla, el aviso permanece pendiente; el correo puede duplicarse tras una caída y conserva Message-ID. Para producción quedan métricas, redrive supervisado y SMTP seguro. No prometo exactly-once distribuido.